Network flows

Capacity and flow conservation describe how a network sends resources from a source to a sink. A residual network records both additional capacity and opportunities to cancel earlier decisions.

Study **augmenting paths, max-flow min-cut and Edmonds–Karp → push–relabel → matching and lower-bound models**.

Prerequisites are graphs, breadth-first search, invariants, and asymptotic analysis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

From flow to proof and modeling

Question	Conditions to check
Is a flow feasible?	Capacity constraints and conservation at intermediate vertices.
How can we improve it or prove optimality?	Residual paths, cut capacity, and max-flow min-cut.
How do local operations converge?	Preflows, heights, and push–relabel invariants.
How do we express new constraints?	Matching, vertex capacities, and flows with lower bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A01 | Maximum flow needs an optimality certificate

Given a directed network $G=(V,E)$, source s, sink t, and finite nonnegative capacities c, maximize the net flow leaving s.

We rebuild the six-vertex example in [NOTE] and finish with a feasible flow of value 11 and a cut of capacity 11. Sources are [CLRS26] § § 26.1–26.2, [MIT04-20], [MIT04-21], and [NOTE] physical pages 38–40.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A02 | Capacity and conservation are separate constraints

Each original edge obeys $0 \le f(e) \le c(e)$. Every v outside $\{s,t\}$ obeys Σincoming $f = \Sigma$outgoing f.

Define $|f| = \Sigma_{(s,v)} f(s,v) - \Sigma_{(v,s)} f(v,s)$. Incoming edges at s must be subtracted. Summing conservation over all internal vertices shows that net source outflow equals net sink inflow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A03 | Choose one model for antiparallel edges

Here f is nonnegative flow on each original edge. MIT04 instead uses skew-symmetric net flow. Our example has no antiparallel original edges.

If u→v and v→u both exist, give each original edge its own paired residual arcs and edge_id. Direction u→v can contain both c(u,v)−f(u,v) of forward capacity and f(v,u) of cancellation capacity. Aggregation is possible only with a defined update allocation. Alternatively subdivide one original edge as u→q→v.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A04 | Residual arcs describe allowed changes

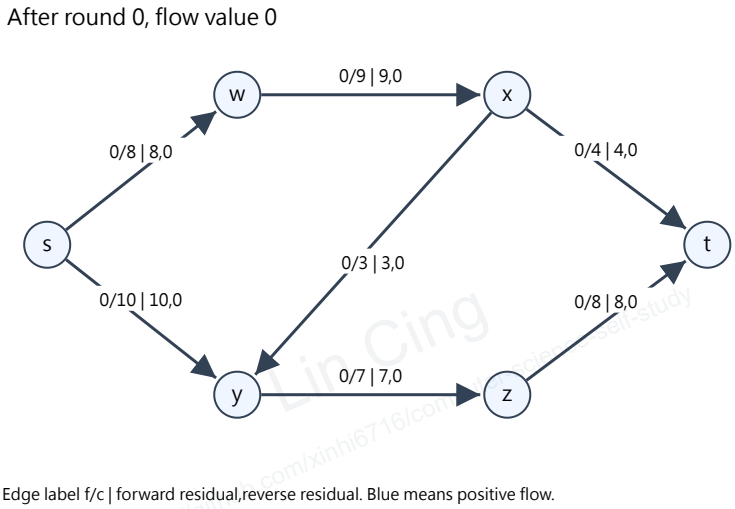
An original u→v edge contributes forward residual c−f and reverse residual f. G_f contains only positive residual arcs.

Sending δ along residual v→u reduces original f(u,v) byδ. It requires no physical reverse transport edge. A missing original v→u edge does not forbid cancellation. Preserve paired reverse arcs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A05 | The six-vertex network starts at zero flow

Capacities are sw=8, sy=10, wx=9, xy=3, yz=7, xt=4, zt=8. Initially every f is 0.



Each label is f/c | forward residual,reverse residual. Arrows show original edges; the final number states the available reverse residual.

A09-A06 | Augmentation preserves feasibility

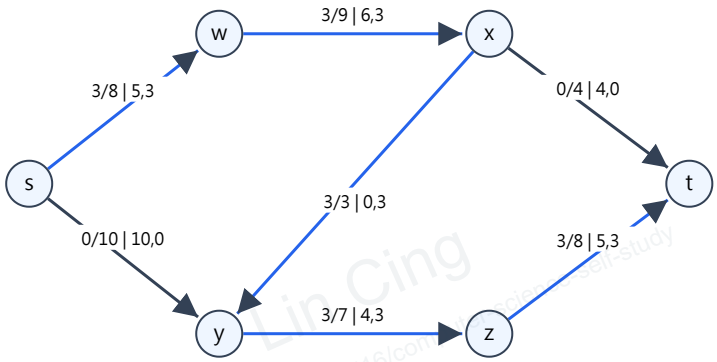
Choose a simple s→t path P in G_f and δ=min_{e∈P}r(e)>0. Addδ on original forward edges and subtractδ on reversed original edges.

The residual bound ensures capacity feasibility. Each internal path vertex receives one incoming and one outgoing flow change, so conservation remains true. Source net outflow and sink net inflow increase byδ. Thus the new value is |f|+δ, and at least one residual arc is saturated.

A09-A07 | Round 1 deliberately chooses a long path

Choose P_1=s→w→x→y→z→t. Residuals are 8,9,3,7,8, so δ_1=3 and the flow value is 3.

After round 1, flow value 3



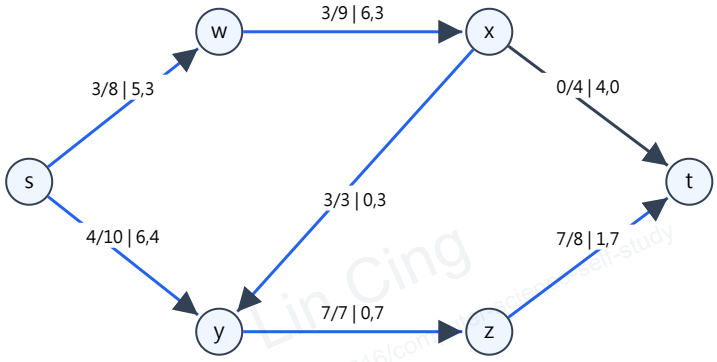
Edge label f/c | forward residual,reverse residual. Blue means positive flow.

Edge xy is saturated and creates reverse residual y→x of 3. This is a valid Ford–Fulkerson choice, not an Edmonds–Karp choice.

A09-A08 | Round 2 fills yz

Choose P_2=s→y→z→t. Residuals are 10,4,5, so δ_2=4 and the value becomes 7.

After round 2, flow value 7

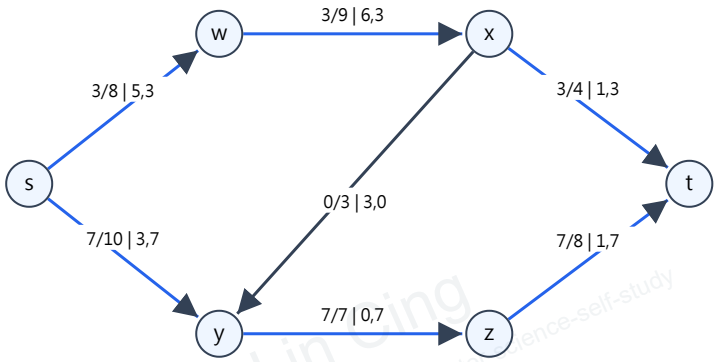


Edge label f/c | forward residual,reverse residual. Blue means positive flow.

At x, inflow 3 equals outflow 3. At y, inflow 3+4 equals outflow 7. Searching only unused original forward capacity would miss the next cancellation.

A09-A09 | Round 3 cancels an earlier decision

After round 3, flow value 10



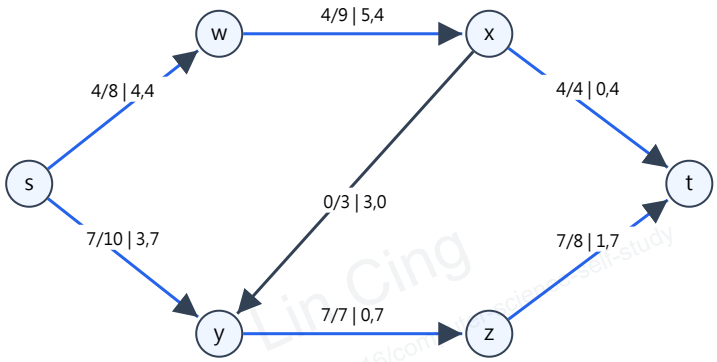
Edge label f/c | forward residual,reverse residual. Blue means positive flow.

Choose P_3=s→y→x→t. Residuals are 6,3,4, so δ_3=3. The arc y→x reverses xy and reduces f(xy) from 3 to 0. Value becomes 10. Now x has inflow 3=outflow 3 and y has inflow 7=outflow 7. Total value increases even though one original edge loses flow.

A09-A10 | Round 4 and final conservation

Choose P_4=s→w→x→t. Residuals are 5,6,1, so δ_4=1. The final value is 11.

After round 4, flow value 11



Edge label f/c | forward residual,reverse residual. Blue means positive flow.

At w,4 enters and 4 leaves; at x,4 enters and 4 leaves; at y and z,7 enters and 7 leaves. Source and sink totals are both 4+7=11. Final edge flows in order sw,sy,wx,xy,yz,xt,zt are (4,7,4,0,7,4,7).

A09-A11 | Audit every round

Round	sw	sy	wx	xy	yz	xt	zt	Value
0	0	0	0	0	0	0	0	0
1	3	0	3	3	3	0	3	3
2	3	4	3	3	7	0	7	7
3	3	7	3	0	7	3	7	10
4	4	7	4	0	7	4	7	11

For each row verify w:sw=wx; x:wx=xy+xt; y:sy+xy=yz; z:yz=zt. check.py audits all rounds and all 16 s–t cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A12 | Cut capacity differs from net cut flow

A cut (S,T) has $s\in S$, $t\in T$, $T=V\setminus S$.

$c(S,T)=\Sigma_{S\to T}c$ counts only original outgoing capacities. Net cut flow is $f(S,T)=\Sigma_{S\to T}f-\Sigma_{T\to S}f$.

Sum outgoing minus incoming flow over vertices in S. Internal edges cancel and conservation removes internal vertices, leaving $|f|=f(S,T)$. Capacity and nonnegativity then give $|f|\leq c(S,T)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A13 | Prove both directions of max-flow min-cut

If $|f|=c(S,T)$ for a cut, every feasible g has $|g| \leq c(S,T)=|f|$. Thus f is maximum and that cut is minimum.

Conversely, maximum f has no augmenting path, since augmentation would increase its value. Let S be the vertices reachable from s in G_f. Then t is outside S. Every original $S \to T$ edge is saturated, or reachability would cross the cut. Every original $T \to S$ edge carries zero flow, or its reverse residual would cross the cut. Hence $|f|=f(S,T)=c(S,T)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A14 | The stopping certificate in the example

The final residual reachable set is S={s,w,x,y}, with T={z,t}. Forward residual on sw,sy,wx,xy permits reachability within S, but yz and xt have zero forward residual.

The cut capacity is c(yz)+c(xt)=7+4=11. No positive flow returns from T to S. A feasible value 11 and an upper bound 11 prove maximum flow=minimum cut=11. The spare unit on zt alone cannot carry additional source flow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A15 | Integral Ford–Fulkerson terminates

```
f[e] = 0 for each original edge e
while find_path(G_f, s, t) returns P:
    delta = min(residual[arc] for arc in P)
    update each paired original edge along P by +delta or -delta
return f, reachable(G_f, s)
```

Initialize zero flow and repeatedly augment by a path bottleneck. Integer capacities keep all residuals and δ integral, so $\delta \geq 1$. A finite cut bounds the flow value, giving at most $F^* = |f^*|$ augmentations.

With $O(E)$ search and update, time is $O(E(1+F^*))$, commonly $O(EF^*)$, plus $O(V+E)$ initialization. This is pseudopolynomial because the numeric F^* may be exponential in the number of capacity input bits.

A09-A16 | A constant-size network can need 2 U rounds

Give su,ut,sv,vt capacity U and uv capacity 1. Alternate s→u→v→t and s→v→u→t. Each δ is 1, with the second path canceling uv. Reaching maximum 2 U takes 2 U augmentations.

Rational capacities can be scaled by a common denominator, ensuring termination without a polynomial bit-length bound. With arbitrary real capacities and arbitrary path choices, Ford–Fulkerson may not terminate. Strict increase alone proves neither finite termination nor an optimal limit.

A09-A17 | Edmonds–Karp chooses a BFS path

Each iteration uses BFS on positive residual arcs to find an s→t path with the fewest edges. It does not maximize path capacity or bottleneck.

On this example choose s→w→x→t with δ=4, then s→y→z→t with δ=7. Two augmentations give 11. These choices differ from the earlier four-round Ford–Fulkerson trace. A general implementation still needs paired reverse arcs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A18 | Why BFS distances never decrease

Let d, d' be source hop distances before and after an augmentation, using ∞ for unreachable vertices. Assume a decrease. Along a new shortest path choose the first decreased vertex v and predecessor u. Thus $d(u) \le d'(u) = d'(v) - 1$.

If (u,v) existed before, $d(v) \le d(u) + 1 \le d'(v)$, a contradiction. Otherwise it was created as the reverse of (v,u) on the old shortest augmenting path. Then $d(u) = d(v) + 1$, so $d(v) = d(u) - 1 \le d'(v) - 2$, again a contradiction. All distances are nondecreasing; an unreachable vertex stays unreachable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A19 | A critical arc must rise two levels before recurring

Each augmentation saturates a critical residual arc (u,v) with $r(u,v)=\delta$ and $d(v)=d(u)+1$. The arc disappears.

Before it can be critical again, an augmentation must traverse (v,u) to restore capacity. At that time $d_{new}(u)=d_{new}(v)+1\geq d_{old}(v)+1=d_{old}(u)+2$. Monotonicity preserves this increase. Finite distances on augmenting paths are at most $V-1$, so each oriented edge_id has only $O(V)$ critical events.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A20 | Count events to obtain the time bound

There are at most $2E$ residual arc directions. Every augmentation has a critical event, and each direction has $O(V)$ events. There are $O(VE)$ augmentations.

BFS and path updates cost $O(V+E)$ per round. Remove isolated vertices or use the relevant reachable subgraph so relevant $V=O(E)$; then each round is $O(E)$, giving $O(VE^2)$, plus $O(V+E)$ initialization. This bound does not depend on capacity magnitude and also holds for exact real arithmetic; bit-operation costs are separate.

Return both a feasible flow and its residual reachable cut. These certify optimality independently of the path history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P01 | Store excess, then restore conservation

Ford–Fulkerson keeps a feasible flow after each augmentation. Push-relabel allows intermediate vertices to store excess. A push examines one residual edge; a relabel changes one vertex height.

The algorithm and proofs follow CLRS 3e § § 26.4–26.5. The original note scan p38 supplies residual-capacity and cancellation prerequisites, rather than the push-relabel algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P02 | Preflow and excess

For each original edge maintain $0 \le f(u,v) \le c(u,v)$. Define $e(u) = \Sigma_v f(v,u) - \Sigma_v f(u,v)$. A preflow requires $e(u) \ge 0$ for every $u \ne s$. A vertex is active exactly when $u \notin \{s,t\}$ and $e(u) > 0$.

Forward residual capacity is $c - f$; reverse residual capacity is f. This example has no antiparallel original edges. A reverse push decreases the corresponding original flow. The source may have negative excess. The sink may have positive excess but is never discharged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P03 | Height is a certificate, not an exact distance

A valid height function satisfies $h(s)=n$, $h(t)=0$, and $h(u)\le h(v)+1$ on every residual edge (u,v). Heights need not equal current distances to t.

A simple residual s-to-t path would have $k\le n-1$ edges. Summing the inequalities gives $n=h(s)\le h(t)+k=k$, a contradiction. Thus a valid height function always rules out such a path. A preflow may still violate conservation, so this alone does not establish completion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P04 | Initialize by saturating source edges

Set all flows, excesses, and heights to zero; set $h(s)=n$. Saturate each original edge (s,v) and update $e(s)$ and $e(v)$. This is initialization, rather than an ordinary PUSH subject to the height-difference test.

The outgoing source edges are absent from the residual network. Their reverse edges $v\to s$ satisfy $0\le n+1$. All other residual endpoints have height zero. Both the preflow and height invariants hold.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P05 | Push conditions and updates

PUSH(u,v) requires an active u, positive residual capacity, and $h(u)=h(v)+1$. Set $\delta=\min(e(u),c_f(u,v))$. Increase original flow or cancel reverse flow, then subtract δ from $e(u)$ and add δ to $e(v)$.

The bounds on δ preserve capacity and nonnegative excess outside s. A newly created reverse edge $v\to u$ satisfies $h(v)=h(u)-1\le h(u)+1$. A saturating push removes the residual edge. A nonsaturating push empties u. If both occur, classify the push as saturating.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P06 | Relabel only when no admissible edge leaves u

RELABEL(u) requires active u and $h(u)\le h(v)$ for all residual neighbors v, equivalently no admissible outgoing edge. Set $h(u)=1+\min\{h(v):c_f(u,v)>0\}$.

Positive excess implies a positive incoming flow and hence an outgoing reverse residual edge, so the minimum is over a nonempty set. The height strictly increases. Outgoing constraints hold by construction; increasing the target height only relaxes incoming constraints.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P07 | Correctness conditional on termination

An active vertex can push if it has an admissible edge and can relabel otherwise. At termination every intermediate excess is zero, so the preflow becomes a flow. The height invariant excludes a residual s-to-t path, which certifies maximum flow.

This proves correctness if the algorithm terminates. Operation bounds below establish termination separately. Neither argument requires integral capacities or bounds iterations by the flow value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P08 | Height bounds and relabel work

Every active u has a residual path to s. Otherwise take its residual reachable set S excluding s. With no residual edge leaving S, original incoming flow across the cut is zero. Thus $\Sigma_{v \in S} e(v) \leq 0$, contradicting positive excess in S.

A simple path back to s has at most n−1 edges, so $h(u) \leq h(s)+n-1=2n-1$. Heights only increase, and only active vertices relabel. Each vertex relabels O(n) times, for $O(n^2)$ total. A relabel scans N(u) in O(deg(u)); all relabel work is $\Sigma_u O(n \deg(u))=O(nm)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P09 | O(nm) saturating pushes

After saturating u→v, another push v→u must restore residual capacity before the next saturation in the same direction. Initially $h(u)=h(v)+1$. At the reverse push, $h'(v)=h'(u)+1\geq h(u)+1$.

At the next forward push, $h''(u)=h''(v)+1\geq h(u)+2$. Repeated saturation in the same direction forces a height increase of at least two. Each direction saturates O(n) times. There are at most 2m residual directions, giving O(nm) saturating pushes. Each push costs O(1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P10 | Potential pays for nonsaturating pushes

Let $\Phi=\Sigma_{u \text{ active}}h(u)$. Initially $\Phi=0$ and always $\Phi\geq 0$. A nonsaturating push removes u and may add v. Since $h(u)=h(v)+1$, Φ decreases by at least one. Neither s nor t enters the sum.

Each relabel or saturating push increases Φ by O(n). There are $O(n^2)$ relabels and O(nm) saturating pushes, so nonsaturating pushes number $O(n^3+n^2m)$. Removing isolated components irrelevant to s and t gives $n=O(m)$, hence $O(n^2m)$. The bound does not depend on capacity magnitude.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P11 | Operation counts require an implementation

An operation count is not yet a running-time proof. Paired residual arcs support O(1) pushes. Stored excesses and an active container support O(1) active-vertex selection.

A persistent current arc scans N(u); only after reaching the end does u relabel and reset its pointer. Each vertex makes O(n) passes, for O(nm) scans and relabel work. Choosing any active vertex and discharging it is a legal generic schedule. Adding push work gives $O(n^2m)$, with O(n+m) initialization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P12 | Relabel-to-front keeps every intermediate vertex

L is a linked list of all vertices in V−{s,t}, including vertices with zero excess. N(u) lists neighbors of both incoming and outgoing original edges in a fixed order. Entries remain even when their residual capacity is zero. Each current pointer persists across calls.

DISCHARGE(u) repeats until e(u)=0. At current=NIL, relabel and reset to the first neighbor. If current is admissible, push without advancing. Otherwise advance current to the next neighbor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P13 | Move after discharge, then use the new successor

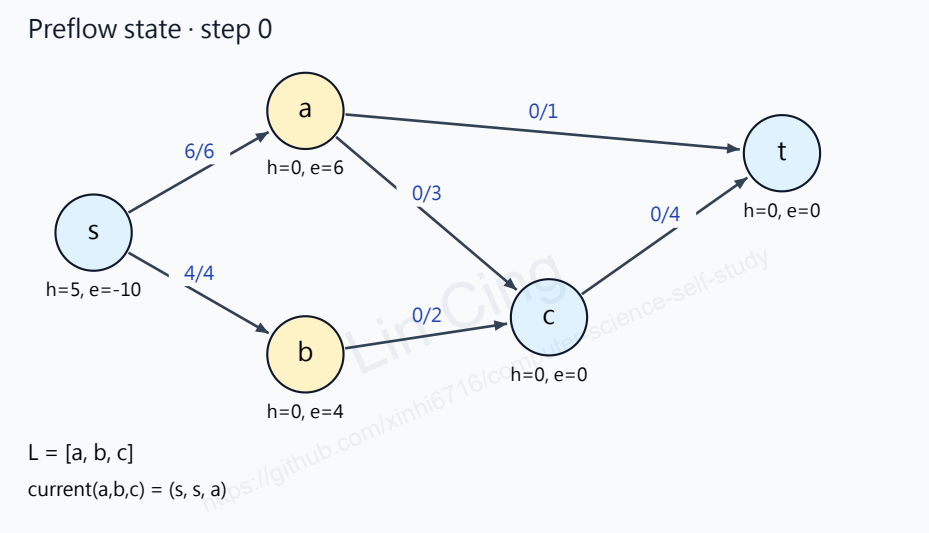
Save old=h(u). After DISCHARGE(u), move u to the front only if h(u)>old. The next vertex is u.next in the modified list. Otherwise advance in the existing order. Zero-excess vertices stay in L.

Why may current persist? Push creates no admissible edge. Relabeling another vertex creates no admissible edge into it. Therefore an outgoing edge skipped during a pass cannot become admissible before u itself relabels. Reaching NIL certifies that relabel applies.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P14 | A fixed five-vertex example

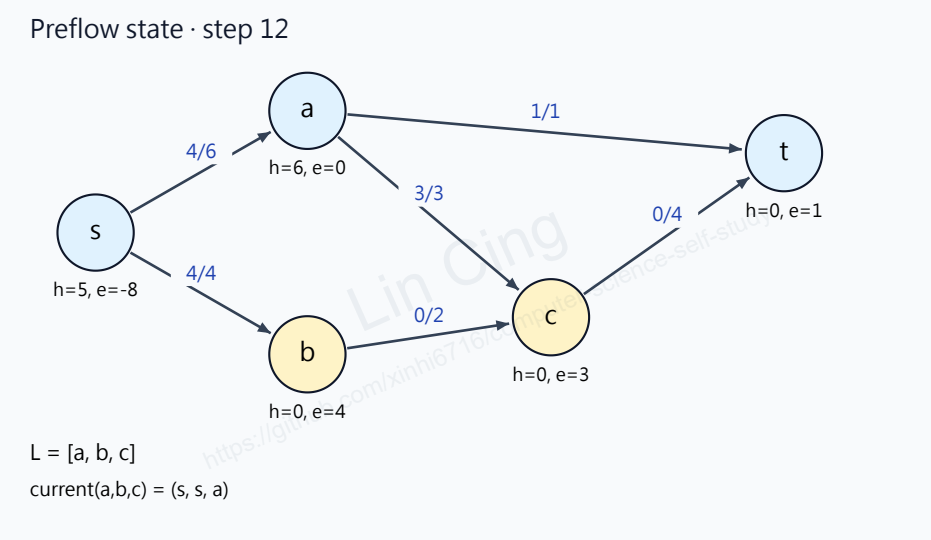
Use V=(s,a,b,c,t), n=5, with capacities sa=6, sb=4, ac=3, at=1, bc=2, ct=4. Fix N(a)=[s,c,t], N(b)=[s,c], N(c)=[a,b,t], and initial L=[a,b,c].



At step 0, h=(5,0,0,0,0), e=(-10,6,4,0,0), current=(s,s,a), and L=[a,b,c].

P15 | a sends flow, then returns excess to s

At height 1, a sends 3 to c and 1 to t. Its remaining excess is 2. Only residual edge a→s remains, so a relabels to 6 and pushes 2 back to s. The source stays at height 5, preserving the required height difference.

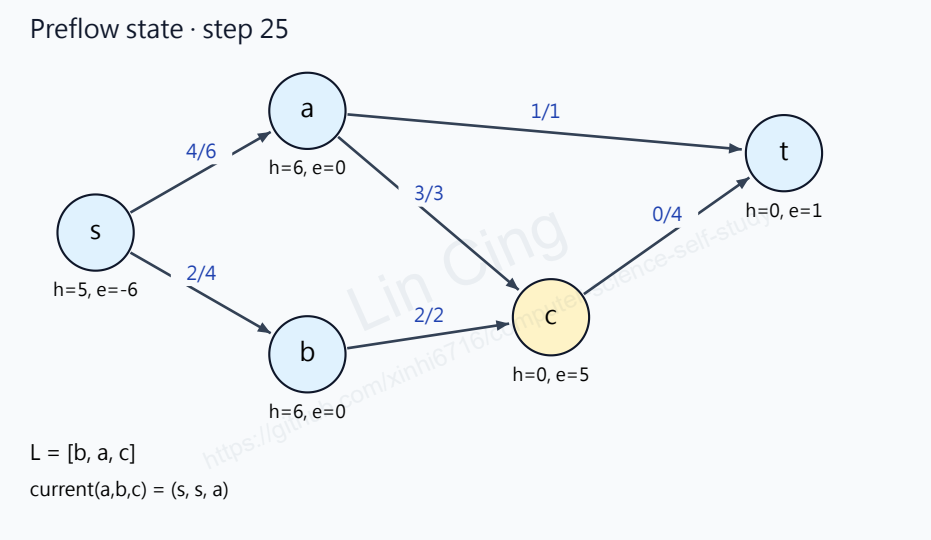


P15 | a sends flow, then returns excess to s (continued)

step	operation	h(s,a,b,c,t)	e(s,a,b,c,t)	current(a,b,c)	L
5	relabel-reset a	5,1,0,0,0	-10,6,4,0,0	s,s,a	a,b,c
7	push a a→c δ=3	5,1,0,0,0	-10,3,4,3,0	c,s,a	a,b,c
9	push a a→t δ=1	5,1,0,0,0	-10,2,4,3,1	t,s,a	a,b,c
11	relabel-reset a	5,6,0,0,0	-10,2,4,3,1	s,s,a	a,b,c
12	push a a→s δ=2	5,6,0,0,0	-8,0,4,3,1	s,s,a	a,b,c
14	move-front a	5,6,0,0,0	-8,0,4,3,1	s,s,a	a,b,c

P16 | b and c move to the front

b sends 2 to c and returns 2 to s, then moves to the front, giving L=[b,a,c]. The following zero-excess discharge of a returns immediately. Then c sends 4 to t, relabels to 7, and returns 1 to a. Moving c gives L=[c,b,a].



P16 | b and c move to the front (continued)

step	operation	h(s,a,b,c,t)	e(s,a,b,c,t)	current(a,b,c)	L
18	relabel-reset b	5,6,1,0,0	-8,0,4,3,1	s,s,a	a,b,c
20	push b b→c δ=2	5,6,1,0,0	-8,0,2,5,1	s,c,a	a,b,c
22	relabel-reset b	5,6,6,0,0	-8,0,2,5,1	s,s,a	a,b,c
23	push b b→s δ=2	5,6,6,0,0	-6,0,0,5,1	s,s,a	a,b,c
25	move-front b	5,6,6,0,0	-6,0,0,5,1	s,s,a	b,a,c
32	relabel-reset c	5,6,6,1,0	-6,0,0,5,1	s,s,a	b,a,c
35	push c c→t δ=4	5,6,6,1,0	-6,0,0,1,5	s,s,t	b,a,c
37	relabel-reset c	5,6,6,7,0	-6,0,0,1,5	s,s,a	b,a,c
38	push c c→a δ=1	5,6,6,7,0	-6,1,0,0,5	s,s,a	b,a,c
40	move-front c	5,6,6,7,0	-6,1,0,0,5	s,s,a	c,b,a

P17 | Final flow, cut, and complete trace

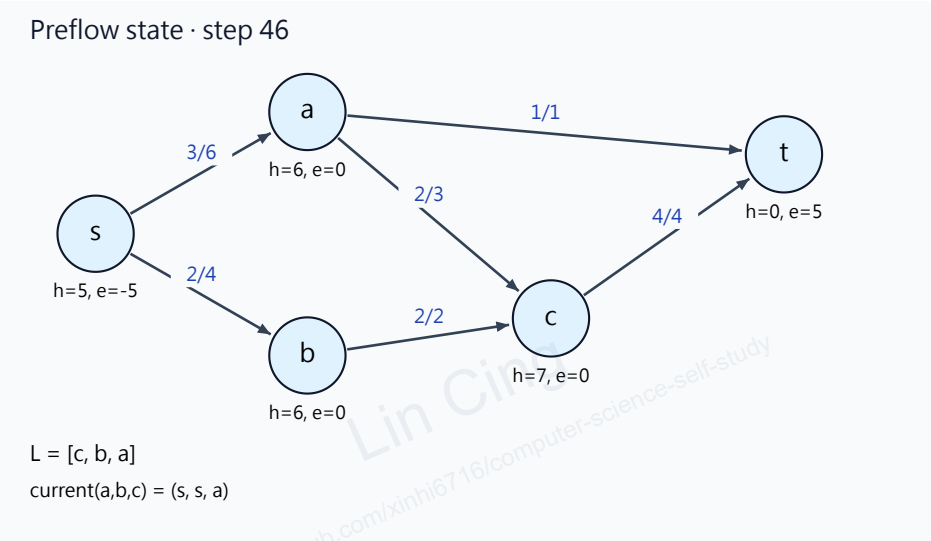
a still has current=s. At height 6 it returns one more unit to s. Finally e=(-5,0,0,0,5), so the flow value is 5. The cut S={s,a,b,c}, T={t} has capacity at+ct=1+4=5, matching the flow.

The chapter solutions give all 47 rows, including pointer advances, discharge boundaries, and list moves. Each figure has the same step number as its table row.

step	operation	h(s,a,b,c,t)	e(s,a,b,c,t)	current(a,b,c)	L
44	push a a→s δ=1	5,6,6,7,0	-5,0,0,0,5	s,s,a	c,b,a
46	terminate	5,6,6,7,0	-5,0,0,0,5	s,s,a	c,b,a

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P17 | Final flow and conservation



The final edge flows are sa=3, sb=2, ac=2, at=1, bc=2, ct=4; the incoming and outgoing flow at each intermediate vertex is equal.

P18 | Why relabel-to-front takes $O(n^3)$

At each outer-loop test, L topologically orders the admissible network on intermediate vertices, and vertices before current u have zero excess. Admissible edges decrease height, so the graph is acyclic. Initially intermediate heights are equal and u is first.

Without a relabel, pushes create no admissible edges and cannot send flow to a predecessor. Discharge empties u, so advancing preserves both claims. If u relabels, after its last relabel it has no admissible incoming edge, and pushes cannot create one. Moving u to the front preserves the order among other vertices; u is empty and is the only predecessor of the next vertex. Both claims hold again. At the end all intermediate excess is zero; valid heights exclude an augmenting path, proving maximum flow.

The claims can fail during discharge. At step 37, $c \to a$ is admissible although c follows a. At step 38, a has excess 1. Moving c at step 40 restores both.

Between relabels there are $O(n)$ DISCHARGE calls; $O(n^2)$ relabels give $O(n^3)$ calls. Relabels, arc advances and saturating pushes cost $O(nm)$. Each discharge has at most one nonsaturating push, which empties u. Total $O(n^3+nm)=O(n^3)$ for a simple graph, $m=O(n^2)$.

Translate requirements into flow constraints

Use nonnegative arc flow x(e) on each original directed edge, with $0 \le x(e) \le c(e)$. If the chapter uses skew-symmetric net flow f, then $f(u,v)=x(u,v)-x(v,u)$. Lower bounds constrain individual arcs, so retain x rather than canceling obligations in opposite directions.

For every reduction, prove that an original solution yields a flow of equal value and that a flow yields an original solution. Discrete choices also require integral flows.

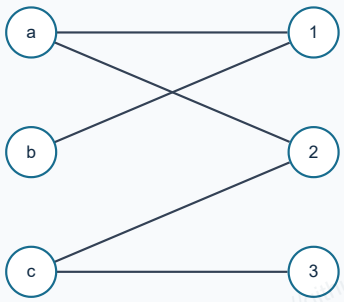
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Build a network for bipartite matching

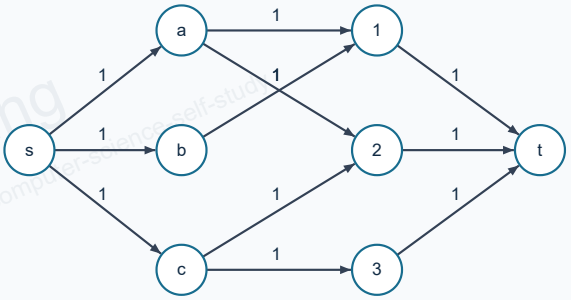
Given $G=(L \cup R, E)$, with disjoint sides and every edge crossing the partition, add s,t. Direct each original edge from L to R. Add $s \to u$ for $u \in L$ and $v \to t$ for $v \in R$. Give every arc capacity 1.

Example $L=\{a,b,c\}$, $R=\{1,2,3\}$, $E=\{a1,a2,b1,c2,c3\}$. The network has 8 vertices and 11 arcs. The left panel is the original graph and the right panel is its flow network.

Original bipartite graph



Flow network, all capacities 1



Solution {a2,b1,c3}; flow value 3; cut {s} has capacity 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Map a matching to a flow

Take $M=\{a2,b1,c3\}$. Send one unit along $s \to u \to v \to t$ for every $uv \in M$, and zero on other arcs. Matching edges share no endpoints, so no source or sink arc is used twice.

Each internal vertex receives and sends the same amount. Capacity and conservation hold. Total flow across $L \to R$ is $|M|=3$. The cut $S=\{s\}$ has capacity 3, proving both the flow and matching optimal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Map an integral flow to a matching

Given an integral flow x, choose $M=\{uv\in E:x(u,v)=1\}$. Capacity 1 on $s\to u$ and conservation allow at most one outgoing arc with positive integral flow at u. Capacity 1 on $v\to t$ gives the symmetric condition at v. Thus M is a matching and $|M|=|x|$.

Starting Ford–Fulkerson at zero preserves integral residual capacities, bottlenecks, and flows when all capacities are integers. This proves existence of an integral maximum flow. It does not make every maximum flow integral. In $K_{2,2}$, flow 1/2 on each middle arc is a maximum flow of value 2, but its positive arcs are not a matching.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A cut explains matching failure

Let $L=\{a,b,c\}$, $R=\{1,2,3\}$, and $E=\{a1,b1,c2,c3\}$. Both a and b can use only vertex 1. Matching $\{a1,c2\}$ has size 2.

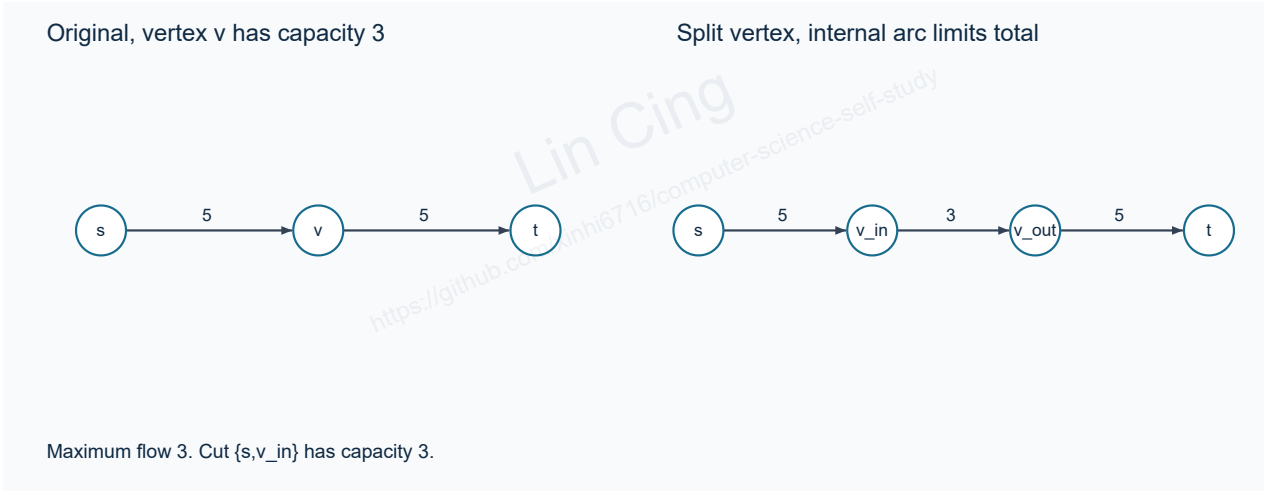
For $S=\{s,a,b,1\}$, the only outgoing cut arcs are $s\to c$ and $1\to t$, each of capacity 1. The cut proves that the maximum matching size is 2. Set $A=\{a,b\}$ has $|N(A)|=1<2$, explaining why L cannot be fully matched. A maximal matching need not be maximum.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vertex capacity limits total throughput

The original network has s→v and v→t of capacity 5, but v can process only 3 units per time unit. Limiting every incident edge separately does not generally bound total throughput when several arcs enter v. The required constraint is $\Sigma_u x(u,v) \leq b(v)$.

Split v into v_in,v_out, joined by an internal arc of capacity b(v)=3. Redirect incoming arcs to v_in and outgoing arcs from v_out. The example has maximum flow 3, certified by cut {s,v_in}.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prove vertex splitting in both directions

Copy each original flow to its redirected arc. Set the internal arc flow to total inflow at v. Original conservation and vertex capacity make this transformed flow feasible. Conversely, conservation at v_in and v_out equates internal arc flow with total inflow and outflow. Merging the pair recovers the original feasible flow.

Splitting all n vertices gives 2n vertices and m+n arcs. Splitting only k constrained internal vertices gives n+k vertices and m+k arcs. To limit total throughput at a source or sink, split it too and attach the external source to s_in and the external sink after t_out.

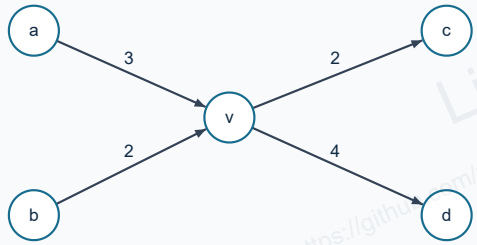
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Add a supersource and supersink

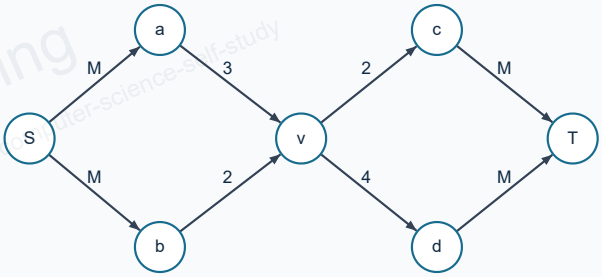
The original problem has sources a,b, sinks c,d, and transit vertex v. Capacities are a→v=3, b→v=2, v→c=2, and v→d=4. Original sources have no incoming arcs and original sinks have no outgoing arcs.

Add S→a,S→b,c→T,d→T. Use M=11, the sum of original capacities. Send 2 on a→v→c, 1 on a→v→d, and 2 on b→v→d, totaling 5. Cut {S,a,b} has capacity 3+2=5 and proves optimality.

Original multisource network



Supernodes, connector capacity M=11



Flow value 5, with 2 delivered to c and 3 to d.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum total flow differs from quota feasibility

Extend an original solution by setting S→a to the net outflow of a and c→T to the net inflow of c. Every original vertex then conserves flow. Removing the supernodes recovers the same total value. This proof requires nonnegative net production at sources and nonnegative net consumption at sinks.

If a,b must supply 3,2 and c,d must receive 2,3, give the new arcs capacities 3,2,2,3 and test for value P=5. Total new source and sink capacity are both 5, so value 5 means every quota arc is saturated. The previous example is feasible. If sink quotas become 3,2, arc v→c still has capacity 2. Cut {S,a,b,v,d} has capacity 2+2=4, proving quota infeasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Choose finite infinity from a proved bound

In a maximum-flow reduction, new arcs must not reduce the original optimum. With finite nonnegative original capacities, $M=\Sigma_e c(e)$ is safe, though possibly loose. A proved upper bound on original throughput also works, such as $3+2=5$ here. A guessed constant is not a general argument.

A demand-feasibility auxiliary network needs to send only D units, so an artificial connector that should not restrict this routing may use capacity D. If a cut argument must strictly exclude artificial arcs, choose M above a known candidate-cut upper bound, such as $U+1$ for an integral model. $M \geq U$ preserves the optimum but can allow a tied artificial cut.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lower bounds create vertex imbalances

A feasible circulation conserves flow at every vertex and satisfies $\ell(e) \leq x(e) \leq u(e)$, with $0 \leq \ell \leq u$. Set $y(e)=x(e)-\ell(e)$, giving capacity $u-\ell$. Define $d(v)=\Sigma_{in}\ell-\Sigma_{out}\ell$.

Substitute into conservation at each vertex. Equation $\Sigma_{in}(y+\ell)=\Sigma_{out}(y+\ell)$ gives $\Sigma_{out}y-\Sigma_{in}y=d(v)$. Thus $d(v)>0$ requires net y outflow $d(v)$, while $d(v)<0$ requires net y inflow $-d(v)$. Each lower bound contributes once positively at its head and once negatively at its tail, so $\Sigma_v d(v)=0$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Derive supernode directions from conservation

Add source S* and sink T*. For d(v)>0, add S*→v of capacity d(v). For d(v)<0, add v→T* of capacity −d(v). Let D=Σ_{d>0}d=Σ_{d<0}(−d).

With demand arcs saturated, a positive-d vertex satisfies Σ_in y+d=Σ_out y. A negative-d vertex satisfies Σ_in y=Σ_out y−d. A zero-d vertex has ordinary conservation. All three cases give Σ_out y−Σ_in y=d.

Feasibility is equivalent to auxiliary maximum-flow value D, or saturation of every arc leaving S*. Since total capacity into T* also equals D, those arcs are saturated too. Remove supernodes and recover x=ℓ+y.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

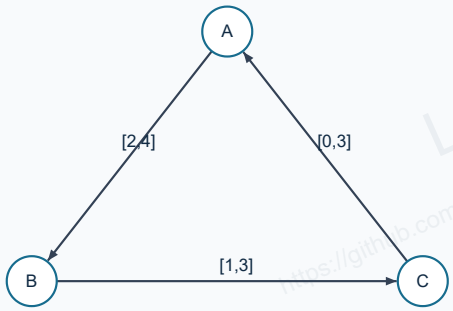
A complete feasible lower-bound example

Original arcs are A→B:[2,4], B→C:[1,3], C→A:[0,3], with brackets denoting [ℓ,u]. Imbalances are d(A)=0−2=−2, d(B)=2−1=1, d(C)=1−0=1, and D=2.

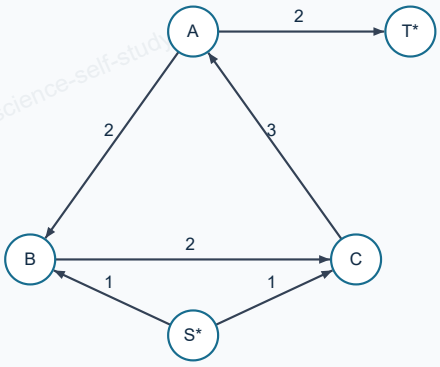
New capacities are 2,2,3. Add S*→B=1, S*→C=1, A→T*=2. Send 1 along S*→B→C→A→T*, then 1 along S*→C→A→T*. This yields y=(0,1,2), hence x=(2,2,2).

Each original vertex receives 2 and sends 2, and every lower and upper bound holds.

Original circulation with bounds



Auxiliary network, capacities u−ℓ



y=(0,1,2), x=(2,2,2); all demand arcs are saturated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An infeasible example needs a deficient cut

Keep the previous example but change C→A to [0,1]. Imbalances and D remain unchanged. Path S*→C→A→T* carries 1 and saturates C→A.

Take auxiliary cut S={S*,B,C}, with complement {A,T*}. Its only outgoing arc is C→A, of capacity 1. Thus maximum flow is at most 1<D=2. In the original network, A must send at least 2 but can receive at most 1, contradicting conservation. Nonnegative remaining capacities alone do not prove lower-bound feasibility.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Convert lower-bounded s-t flow to circulation

First restrict attention to s-t flows with nonnegative value. Add a return arc t→s with bounds [0,M] so every vertex conserves flow. Its flow equals the original s-t value. To test an exact value F, use [F,F] and apply the demand reduction.

Example s→a:[1,3], a→t:[1,2], with F=2. Adding t→s:[2,2] gives d(s)=1, d(a)=0, d(t)=−1. Send 1 along S*→s→a→t→T*. Recover flow 2 on both original arcs and 2 on the return arc. F=3 is infeasible because a→t has upper bound 2.

For feasibility without a fixed value, M=Σ original upper bounds is safe. A single nonnegative return arc does not represent allowed negative flow values; those require another model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Validate the recovered solution

Check original choices or quotas first, then bounds on every original arc and conservation at every original vertex. Provide an equal-value cut, saturated demand arcs, or a deficient cut as a certificate. Use integrality only when transformed capacities are integral and an integral flow is obtained.

General real capacities allow fractional flows. Even integral capacities permit fractional feasible flows and fractional maximum flows. Matching extraction requires an integral maximum flow. Integral lower and upper bounds admit an integral circulation whenever the auxiliary integral-flow construction is feasible.

Sources are CLRS 26.1 and 26.3, with Exercises 26.1-7, 26.2-5, 26.2-6, and 26.3-2. The lower-bound reduction and numerical examples are derived here. check.py verifies examples by enumeration and cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 26, Maximum Flow. Original handwritten algorithms notes (演算法_wjungle), network-flow pages. MIT 6.046J, Fall 2004, Lectures 20 and 21, Maximum Flow and Edmonds–Karp.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study